



第16课时：字符串常用操作

比较 · 查找 · ASCII · 遍历修改

字符串比较、子串查找与提取、字符与ASCII码、字符串遍历与修改



上节课回顾



上节课我们学习了 `string` 类型的基础：

- 声明与初始化： `string s = "Hello";`
- 输入输出： `cin >>` 和 `getline()`
- 长度获取： `size()` / `length()`
- 下标访问： `s[0]`、`s[1]` ...
- 拼接： `+` 和 `+=`

本节课我们将深入字符串的**高级操作**——比较、查找、提取、遍历和修改，让字符串处理能力更上一层楼。

字符串比较——关系运算符



`string` 类型支持使用**关系运算符**进行比较，非常直观：

```
string s1 = "apple";  
string s2 = "banana";  
  
cout << (s1 == s2) << endl;    // 0 (false, 不相等)  
cout << (s1 != s2) << endl;    // 1 (true, 不等)  
cout << (s1 < s2) << endl;     // 1 (true, apple < banana)  
cout << (s1 > s2) << endl;     // 0 (false)  
cout << (s1 <= s2) << endl;    // 1 (true)  
cout << (s1 >= s2) << endl;    // 0 (false)
```

💡 **比较规则：**按字典序（ASCII码顺序）逐字符比较，第一个不同的字符决定结果。

字符串比较的规则



字典序比较的详细规则：

1. 从左到右逐字符比较
2. 如果当前字符不同，ASCII码**小**的字符串**更小**
3. 如果所有字符都相同，但一个字符串更长，短的那个**更小**
4. 如果完全相同，则**相等**

```
"abc" < "abd"      // 第3个字符 c < d
"abc" < "abcd"     // "abc"是"abcd"的前缀，更短的更小
"abc" == "abc"     // 完全相同
"ABC" < "abc"      // 大写字母ASCII码 < 小写字母
"123" < "abc"      // 数字字符 < 字母字符
```

⚠ **重要：**大写字母 `'A'` ~ `'Z'` (65~90) 的ASCII码**小于**小写字母 `'a'` ~ `'z'` (97~122)，所以
`"Apple" < "apple"`。

字符串比较——compare()函数



string 还提供了 compare() 函数，功能更灵活：

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s1 = "Hello World";
    string s2 = "Hello";

    // 整体比较：返回0(相等)、正数(s1>s2)、负数(s1<s2)
    int result = s1.compare(s2);
    cout << result << endl; // 正数 (s1更长)

    // 比较s1从位置0开始的5个字符 与 s2
    result = s1.compare(0, 5, s2);
    cout << result << endl; // 0 ("Hello" == "Hello")

    // 比较s1从位置6开始的5个字符 与 s2
    result = s1.compare(6, 5, s2);
    cout << result << endl; // 负数 ("World" < "Hello"为false, 实际为正)
    return 0;
}
```

💡 一般情况下用关系运算符即可，compare() 用于需要**比较子串**的场景。

子串查找——find()函数



`find()` 函数用于在字符串中**查找子串**的位置，返回**第一次出现**的下标。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello World Hello C++";
    string target = "World";

    // 查找子串
    size_t pos = s.find(target);
    if (pos != string::npos) {
        cout << "找到, 位置: " << pos << endl;
        // 输出: 找到, 位置: 6
    } else {
        cout << "未找到" << endl;
    }

    // 从指定位置开始查找
    pos = s.find("Hello", 7); // 从位置7开始找
    cout << "第二个Hello位置: " << pos << endl;
    // 输出: 12
    return 0;
}
```

⚠ **注意：**如果未找到，`find()` 返回 `string::npos`（一个特殊值），**不能**直接和 `-1` 比较。



子串查找——rfind()与find_first_of()



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello World Hello C++";

    // rfind(): 从后往前找，返回最后一次出现的位置
    size_t pos = s.rfind("Hello");
    cout << "最后一个Hello位置: " << pos << endl;
    // 输出: 12

    // find_first_of(): 查找第一个匹配指定字符集合的位置
    pos = s.find_first_of("aeiou");
    cout << "第一个元音位置: " << pos << endl;
    // 输出: 1 ('e'在位置1)

    // find_last_of(): 查找最后一个匹配字符集合的位置
    pos = s.find_last_of("lo");
    cout << "最后一个l或o位置: " << pos << endl;
    // 输出: 17
    return 0;
}
```

子串提取——substr()函数



`substr()` 函数用于提取字符串的子串。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello World C++";

    // substr(起始位置, 长度)
    string sub1 = s.substr(0, 5);    // "Hello"
    cout << sub1 << endl;

    // substr(起始位置) — 从该位置到末尾
    string sub2 = s.substr(6);       // "World C++"
    cout << sub2 << endl;

    // 提取单个字符
    string sub3 = s.substr(2, 1);    // "l"
    cout << sub3 << endl;

    // 结合find()使用
    size_t pos = s.find("World");
    if (pos != string::npos) {
        cout << s.substr(pos, 5) << endl; // "World"
    }
    return 0;
}
```

字符与ASCII码



每个字符在计算机中都对应一个**ASCII码**（整数）。C++中，`char` 类型本质上就是一个小整数。

字符	ASCII码	说明
<code>'0' ~ '9'</code>	48~57	数字字符
<code>'A' ~ 'Z'</code>	65~90	大写字母
<code>'a' ~ 'z'</code>	97~122	小写字母
<code>' '</code>	32	空格
<code>'\n'</code>	10	换行

关键关系：

- 大写字母 + 32 = 小写字母 （如 `'A' + 32 = 'a'`）
- 小写字母 - 32 = 大写字母 （如 `'a' - 32 = 'A'`）



ASCII码的常用操作



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    // 1. 查看字符的ASCII码
    char c = 'A';
    cout << (int)c << endl;    // 65

    // 2. ASCII码转字符
    int code = 97;
    cout << (char)code << endl; // a

    // 3. 大写转小写
    char upper = 'H';
    char lower = upper + 32;    // 或 upper - 'A' + 'a'
    cout << lower << endl;    // h

    // 4. 小写转大写
    char lower2 = 'z';
    char upper2 = lower2 - 32;
    cout << upper2 << endl;    // Z

    // 5. 数字字符转数值
    char digit = '7';
    int num = digit - '0';
    cout << num << endl;    // 7
    return 0;
}
```

字符串的遍历



遍历字符串就是**逐个访问每个字符**，与遍历数组类似。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";

    // 方式1: 下标遍历
    for (int i = 0; i < s.size(); i++) {
        cout << s[i] << " ";
    }
    cout << endl;
    // 输出: H e l l o

    // 方式2: 范围for循环 (C++11)
    for (char c : s) {
        cout << c << " ";
    }
    cout << endl;
    // 输出: H e l l o

    // 方式3: auto自动推导
    for (auto c : s) {
        cout << c << " ";
    }
    cout << endl;
    return 0;
}
```

字符串的修改



可以通过 `[]` 下标直接修改字符，也可以使用 `insert()`、`erase()` 等函数。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "Hello";

    // 1. 修改单个字符
    s[0] = 'h';
    cout << s << endl;          // hello

    // 2. insert() 在指定位置插入
    s.insert(5, "!");
    cout << s << endl;          // hello!

    // 3. erase() 删除指定位置的字符
    s.erase(5, 1);               // 从位置5删除1个字符
    cout << s << endl;          // hello

    // 4. replace() 替换
    s.replace(0, 2, "HE");        // 从位置0替换2个字符为"HE"
    cout << s << endl;          // HELLo

    return 0;
}
```

综合例题1：统计元音字母



题目：输入一个字符串，统计其中元音字母（a, e, i, o, u，不区分大小写）的个数。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    getline(cin, s);

    int count = 0;
    for (int i = 0; i < s.size(); i++) {
        char c = s[i];
        if (c >= 'A' && c <= 'Z') c += 32; // 转小写
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            count++;
        }
    }
    cout << "元音字母个数: " << count << endl;
    return 0;
}
// 输入: Hello World
// 输出: 元音字母个数: 3
```

综合例题2：大小写转换



题目：输入一个字符串，将所有小写字母转大写，大写字母转小写，其他字符不变。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    getline(cin, s);

    for (int i = 0; i < s.size(); i++) {
        if (s[i] >= 'a' && s[i] <= 'z') {
            s[i] = s[i] - 32;    // 小写转大写
        } else if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = s[i] + 32;    // 大写转小写
        }
    }
    cout << s << endl;
    return 0;
}
// 输入: Hello World 123
// 输出: hELLO wORLD 123
```

综合例题3：子串查找与提取



题目：输入一个字符串和一个子串，查找子串在字符串中出现的所有位置。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s, sub;
    getline(cin, s);
    cin >> sub;

    size_t pos = 0;
    int count = 0;
    while ((pos = s.find(sub, pos)) != string::npos) {
        cout << "位置: " << pos << endl;
        pos += sub.size(); // 继续从后面查找
        count++;
    }
    if (count == 0) {
        cout << "未找到" << endl;
    } else {
        cout << "共出现 " << count << " 次" << endl;
    }
    return 0;
}
// 输入: abcabcab / abc
// 输出: 位置: 0 / 位置: 3 / 位置: 6 / 共出现 3 次
```

综合例题4：字符串反转



题目：输入一个字符串，将其反转后输出（不使用库函数）。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    getline(cin, s);

    // 双指针反转
    int left = 0, right = s.size() - 1;
    while (left < right) {
        swap(s[left], s[right]);
        left++;
        right--;
    }
    cout << s << endl;
    return 0;
}
// 输入: Hello World
// 输出: dlroW olleH
```

也可以用 `#include <algorithm>` 的 `reverse()` 函数: `reverse(s.begin(), s.end());`

综合例题5：判断回文字符串



题目：输入一个字符串，判断它是否是回文串（正读和反读相同）。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;

    bool isPalindrome = true;
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            isPalindrome = false;
            break;
        }
        left++;
        right--;
    }

    if (isPalindrome) {
        cout << s << " 是回文串" << endl;
    } else {
        cout << s << " 不是回文串" << endl;
    }
    return 0;
}
// 输入: aba
// 输出: aba 是回文串
// 输入: abc
// 输出: abc 不是回文串
```

综合例题6：提取数字



题目：输入一个字符串，提取其中所有的数字字符，组成一个整数并输出。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    getline(cin, s);

    string digits = "";
    for (int i = 0; i < s.size(); i++) {
        if (s[i] >= '0' && s[i] <= '9') {
            digits += s[i];
        }
    }

    if (digits.empty()) {
        cout << "无数字" << endl;
    } else {
        int num = stoi(digits); // string转int
        cout << "提取的数字: " << num << endl;
    }
    return 0;
}
// 输入: abc123def456
// 输出: 提取的数字: 123456
```

字符串常用函数汇总



函数	作用	示例
<code>s.find(sub)</code>	查找子串位置	<code>s.find("lo")</code> → 3
<code>s.rfind(sub)</code>	从后查找子串	<code>s.rfind("l")</code> → 9
<code>s.substr(pos, len)</code>	提取子串	<code>s.substr(0, 3)</code> → "Hel"
<code>s.insert(pos, str)</code>	在位置插入	<code>s.insert(2, "XX")</code>
<code>s.erase(pos, len)</code>	删除子串	<code>s.erase(0, 2)</code>
<code>s.replace(pos, len, str)</code>	替换子串	<code>s.replace(0, 2, "XX")</code>
<code>s.compare(s2)</code>	比较 (返回int)	<code>s.compare("hi")</code>
<code>stoi(s)</code>	string转int	<code>stoi("42")</code> → 42
<code>to_string(n)</code>	数字转string	<code>to_string(42)</code> → "42"

常见错误与调试



错误类型	示例/说明	解决方法
find()返回值判断错误	<code>if (s.find(sub) == -1)</code>	用 <code>string::npos</code> 判断
substr越界	<code>s.substr(10, 5)</code> 当长度不够时	先检查 <code>pos + len <= s.size()</code>
大小写混淆	<code>"Hello" < "apple"</code> 为真 (大写ASCII小)	需要时先统一大小写
insert/erase位置错误	在错误位置插入或删除	打印前后字符串确认变化
stoi转换失败	<code>stoi("abc")</code> 会抛出异常	确保字符串只包含数字

调试技巧： 每次操作后打印字符串状态；用断点检查 `pos` 和 `i` 的值。

知识小结



- ✓ **字符串比较**: 用 `==`、`!=`、`<`、`>` 等关系运算符, 按字典序比较; `compare()` 可比较子串
- ✓ **子串查找**: `find()` 查找首次出现, `rfind()` 查找末次出现, 返回 `string::npos` 表示未找到
- ✓ **子串提取**: `substr(pos, len)` 提取从 `pos` 开始长度为 `len` 的子串
- ✓ **ASCII码**: `char` 本质是整数, `'A'` = 65、`'a'` = 97、`'0'` = 48, 大小写差32
- ✓ **字符串遍历**: 下标循环 `s[i]` 或范围for循环 `for (char c : s)`
- ✓ **字符串修改**: `[]` 修改字符、`insert()` 插入、`erase()` 删除、`replace()` 替换
- ✓ **类型转换**: `stoi()` 字符串转数字, `to_string()` 数字转字符串

下节课预告



下一阶段：函数与结构化编程（第17-19课时）

- 第17课时：函数的定义与调用
- 第18课时：函数进阶——递归入门
- 第19课时：结构体

"字符串是数据的衣服，学会裁剪与缝纫，你就能为数据穿上任何形态。"

谢谢大家！



字符串操作是编程的基本功，勤加练习才能熟能生巧！

Q & A